A computer system has four different types of resources (printer, ROM, hard disk and RAM) and it needs to complete execution of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint). The number of allocated resources, maximum needed resources to complete the execution and total available resources should be taken from the user.

```
┌──(kali㉿kali)-[~]
└─$ cat > banker.c
#include <stdio.h>

int main() {
    int i,j,k,n=5,m=4;
    int alloc[5][4],max[5][4],need[5][4];
    int avail[4],finish[5]={0},safe[5],count=0;

    printf("Enter Allocation Matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&alloc[i][j]);

    printf("Enter Max Matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&max[i][j]);

    printf("Enter Available Resources:\n");
    for(i=0;i<m;i++)
        scanf("%d",&avail[i]);

    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j]=max[i][j]-alloc[i][j];

    while(count<n){
        for(i=0;i<n;i++){
            if(finish[i]==0){
                int flag=0;
                for(j=0;j<m;j++){
                    if(need[i][j]>avail[j]){
                        flag=1;
                        break;
                    }
                }
                if(flag==0){
                    safe[count++]=i;
                    for(k=0;k<m;k++)
                        avail[k]+=alloc[i][k];
                    finish[i]=1;
                }
            }
        }
    }

    printf("Safe Sequence:\n");
    for(i=0;i<n;i++)
        printf("P%d ",safe[i]);

    return 0;
}
^C
```

OUTPUT:-

```
File  Actions  Edit  View  Help
        for(i=0;i<n;i++){
            if(finish[i]==0){
                int flag=0;
                for(j=0;j<m;j++){
                    if(need[i][j]>avail[j]){
                        flag=1;
                        break;
                    }
                }
                if(flag==0){
                    safe[count++]=i;
                    for(k=0;k<m;k++)
                        avail[k]+=alloc[i][k];
                    finish[i]=1;
                }
            }
        }
    }

    printf("Safe Sequence:\n");
    for(i=0;i<n;i++)
        printf("P%d ",safe[i]);

    return 0;
}
^C

┌──(kali㉿kali)-[~]
└─$ ls
banker.c  Desktop  Documents  Downloads  Music  om  Pictures  Public  Templates  Videos

┌──(kali㉿kali)-[~]
└─$ gcc banker.c

┌──(kali㉿kali)-[~]
└─$ ls
a.out  banker.c  Desktop  Documents  Downloads  Music  om  Pictures  Public  Templates  Videos

┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter Allocation Matrix:
0 1 0 3
2 0 0 1
3 0 2 1
2 1 1 0
0 0 2 2
Enter Max Matrix:
7 5 3 4
3 2 2 2
9 0 2 2
2 2 2 2
4 3 3 3
Enter Available Resources:
3 3 2 2
Safe Sequence:
P1 P3 P4 P0 P2

┌──(kali㉿kali)-[~]
└─$
```